

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OPERATING SYSTEMS COURSE CODE: CSA04

COURSE OUTCOME COVERED

CO2: Analyze process management and deadlock handling techniques to improve CPU utilization and system performance(BL3-BL4)

Assessment Tool Weightage: 30%

QUESTIONS: 10

Total Marks:50

Annexure A

Scenario based Questions

Code of Conduct:

I Shaik Rafiqhuddin (192525129) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: 



QUESTION 1

Real-Time Ticket Booking — CPU Scheduling

10 MARKS

Original Question

Question 1 — CPU Scheduling in a Real-Time Ticket Booking System

A railway ticket booking system experiences heavy load during festival seasons. The system handles three categories of processes:

- P1 – Payment gateway transactions (High priority)
- P2 – Seat availability checks (Medium priority)
- P3 – Ticket confirmation and printing (Low priority)
- Burst times vary from 2 ms to 20 ms.

Recently, customers complained that ticket confirmations are taking too long.

Tasks

1. Identify the most suitable scheduling algorithm (FCFS, SJF, Priority, RR) for this scenario.
2. Justify why your chosen method improves CPU utilization and reduces customer wait time.
3. Provide a sample execution order for the above categories.
4. Discuss one drawback of this scheduling method in real-time systems.

(10 Marks)

ANSWER

Aim

To identify and justify the most suitable CPU scheduling algorithm for a railway ticket booking system handling payment, seat-availability, and confirmation/printing processes under peak festival load.

Problem Understanding

Three classes of work compete for the CPU with very different urgency: payment transactions must complete quickly and correctly (financial integrity, high priority), seat checks are moderately time-sensitive (medium priority), and ticket confirmation/printing is treated as low

priority. Customers are complaining specifically about confirmation delays, which signals that whatever scheduling is currently used is letting low-priority work fall far behind.

Given Data

Process Category	Priority	Typical Burst Time
P1 – Payment gateway	High (1)	2–6 ms
P2 – Seat availability check	Medium (2)	3–10 ms
P3 – Ticket confirmation & printing	Low (3)	8–20 ms

Assumptions

Burst times within each category are assumed as shown above (spanning the given 2–20 ms range); multiple instances of each category arrive concurrently during a festival-season burst.

Theory & OS Concept Used

Priority Scheduling

Priority Scheduling dispatches the process with the highest priority (lowest priority number in this document) among those ready.

Working Principle: Ties are broken by arrival order; a preemptive variant interrupts a lower-priority running process the instant a higher-priority one arrives.

Advantages: Fast service for urgent/critical work; Flexible urgency modelling.

Disadvantages: Starvation of low-priority processes without aging.

Applications: Real-time and emergency systems, tiered service.

Detailed Explanation

Preemptive Priority Scheduling (implemented internally as a Multilevel Queue: a high-priority queue for payments, a medium queue for seat checks, and a low-priority queue for confirmations, each served round-robin within its level) is the most suitable choice. It guarantees that time-critical, revenue-affecting payment transactions are never delayed behind bulk printing jobs, while still allowing seat checks to be served promptly.

Step-by-Step Solution

1. Classify every incoming process into one of the three priority queues based on its category (payment/seat-check/confirmation).

2. Whenever the CPU is free, the scheduler always dispatches from the highest non-empty priority queue (Payment → Seat Check → Confirmation).
3. Within the same queue, processes are served FCFS (or Round Robin if many requests of the same type arrive together) to keep things fair among peers.
4. Apply aging to the Confirmation queue: any confirmation job waiting longer than a threshold (e.g., 50 ms) has its effective priority raised so it is eventually guaranteed service, directly fixing the customer complaint.

Process Analysis / Sample Execution Order

Assume six representative processes arrive together at the start of a burst:

Process	Category	Burst (ms)	Priority
Pay1	Payment	4	1
Pay2	Payment	3	1
Seat1	Seat Check	6	2
Seat2	Seat Check	5	2
Conf1	Confirmation	18	3
Conf2	Confirmation	12	3

Gantt Chart — Sample Execution Order (Priority-based)

Pay1	Pay2	Seat1	Seat2	Conf1	Conf2
0-4	4-7	7-13	13-18	18-36	36-48

Execution order: Pay1 → Pay2 → Seat1 → Seat2 → Conf1 → Conf2. Notice that, exactly as customers reported, both confirmation jobs are pushed to the very end (18 ms and 36 ms of pure waiting) under strict priority — this is why the aging rule in Step 4 is essential.

Scheduling Analysis

 **Key Concept:** Priority Scheduling alone explains the complaint (confirmations always run last); Priority + Aging fixes it by bounding the worst-case wait for low-priority ticket confirmation jobs.

Performance Analysis

Payment average waiting time = $(0+4)/2 = 2$ ms and Seat-check average waiting time = $(7+13)/2 = 10$ ms — both very responsive. Confirmation average waiting time = $(18+36)/2 = 27$ ms under

plain priority, which explains the complaint; with aging this is bounded to the configured aging threshold instead of growing indefinitely under sustained load.

Limitations

One drawback in real-time systems: strict Priority Scheduling can indefinitely starve low-priority processes (exactly the ticket-confirmation symptom observed) if aging or a fairness mechanism is not implemented; tuning the aging threshold also adds configuration complexity and, if set too aggressively, can itself delay payment processing.

Real-world Applications

Airline and railway reservation systems, banking transaction queues, and any e-commerce checkout pipeline that must separate payment-critical paths from bulk/background operations.

Result

Preemptive Priority Scheduling (as a 3-level Multilevel Queue with aging) is recommended: Payment > Seat Check > Confirmation, with aging applied to the Confirmation queue.

Conclusion

This approach improves CPU utilization by ensuring high-value, short transactions are never stuck behind long jobs, while aging protects ticket confirmation and printing from the starvation that caused the original customer complaints — giving the railway booking system both responsiveness and fairness under festival-season load.



QUESTION 2

Banking Transaction Server — Deadlock

10 MARKS

Original Question

Question 2 — Deadlock in a Banking Transaction Server

A bank server handles the following operations simultaneously:

- T1: Transfers money between accounts
- T2: Updates passbook entries
- T3: Generates monthly statements

Due to simultaneous locking of Account Database (A), Customer Info Database (B), and Transaction Logs (C), the system enters a state where:

- T1 holds A and requests B
- T2 holds B and requests C
- T3 holds C and requests A

Tasks

1. Draw the resource request situation (text-based RAG).
2. Identify whether a deadlock has occurred and justify your answer.
3. Recommend one specific technique (avoidance/prevention/recovery) and explain how it fixes the issue.
4. Suggest a redesign for the locking order to avoid future deadlocks.

(10 Marks)

ANSWER

Aim

To model the bank server's lock contention as a Resource Allocation Graph, determine whether a deadlock exists, and design a fix.

Problem Understanding

Three transactions each hold one single-instance resource and request another, in a pattern that visually forms a cycle: $T1 \rightarrow (\text{needs}) B \rightarrow (\text{held by}) T2 \rightarrow (\text{needs}) C \rightarrow (\text{held by}) T3 \rightarrow (\text{needs}) A \rightarrow (\text{held by}) T1$.

Given Data

Transaction	Holds	Requests
T1	A (Account Database)	B (Customer Info Database)
T2	B (Customer Info Database)	C (Transaction Logs)
T3	C (Transaction Logs)	A (Account Database)

Theory & OS Concept Used

A Resource Allocation Graph (RAG) represents processes and resources as nodes; an assignment edge (Resource $\rightarrow$ Process) shows a resource currently held, and a request edge (Process $\rightarrow$ Resource) shows a resource being waited for. If every resource in the graph has a single instance, a cycle in the RAG is both a necessary and sufficient condition for deadlock.

Detailed Explanation / Resource Allocation Graph (text-based)

Process	Relationship	Resource	Edge Type
T1	holds $\rightarrow$	A	Assignment edge (Resource $\rightarrow$ Process)
T1	requests $\rightarrow$	B	Request edge (Process $\rightarrow$ Resource)
T2	holds $\rightarrow$	B	Assignment edge (Resource $\rightarrow$ Process)
T2	requests $\rightarrow$	C	Request edge (Process $\rightarrow$ Resource)
T3	holds $\rightarrow$	C	Assignment edge (Resource $\rightarrow$ Process)
T3	requests $\rightarrow$	A	Request edge (Process $\rightarrow$ Resource)

Wait-for Graph (derived from the RAG)

Waiting Process	Waits For (Holder of needed resource)
T1	T2
T2	T3
T3	T1

Tracing the graph: $A \rightarrow T1 \rightarrow B \rightarrow T2 \rightarrow C \rightarrow T3 \rightarrow A$ closes into a cycle, and the equivalent Wait-for Graph $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$ is also a cycle.

Deadlock Analysis

Yes, a deadlock has occurred. Justification — all four Coffman conditions hold simultaneously:

Coffman Condition	Present in this Scenario?
Mutual Exclusion	Yes — A, B, C are locked databases usable by only one transaction at a time
Hold and Wait	Yes — each transaction holds one lock while requesting another (e.g., T1 holds A, requests B)
No Preemption	Yes — none of the database locks can be forcibly taken away from a transaction
Circular Wait	Yes — $T1 \rightarrow T2 \rightarrow T3 \rightarrow T1$ forms a closed cycle
⚠ Important: Because every resource here is single-instance, the presence of the cycle alone is sufficient proof of deadlock — no further reduction/safety check is needed.	

Recommended Technique

Recommended technique: Deadlock Prevention via Resource Ordering (breaking the Circular Wait condition). A global, fixed lock-acquisition order is imposed on all resources: A (Account DB) $\rightarrow$ B (Customer Info DB) $\rightarrow$ C (Transaction Logs). Every transaction, regardless of its business purpose, must request locks strictly in this increasing order.

Why this fixes the issue: with a total order enforced, it becomes structurally impossible for a cycle to form, because a transaction can never be found holding a “later” resource in the order while requesting an “earlier” one. Specifically, T3 (which needed C before A) would now be required to request A before C, eliminating the $T3 \rightarrow A$ edge that closed the cycle.

Redesigned Locking Order

Transaction	Old (Deadlock-Prone) Order	New (Ordered) Locking Sequence
T1	A then B	A then B (unchanged — already in order)
T2	B then C	B then C (unchanged — already in order)
T3	C then A	A then C (reversed to respect global order $A \rightarrow B \rightarrow C$)

With T3 redesigned to acquire A before C, T3 can never hold C while waiting on A, so the cycle $A \rightarrow T1 \rightarrow B \rightarrow T2 \rightarrow C \rightarrow T3 \rightarrow A$ can no longer close.

Limitations

Resource ordering requires all developers/modules to strictly follow the agreed order; adding a new shared resource later requires re-auditing the ordering across the whole system, and it can force some transactions to hold a lock slightly longer than functionally necessary while waiting to acquire a later-ordered resource.

Real-world Applications

Database transaction managers (e.g., two-phase locking with ordered lock acquisition), operating-system kernel lock hierarchies, and distributed banking/ledger systems.

Result

The scenario is a confirmed deadlock (circular wait among T1, T2, T3). Deadlock Prevention via a global resource-acquisition order ($A \rightarrow B \rightarrow C$) is recommended, with T3's lock order redesigned from $C \rightarrow A$ to $A \rightarrow C$.

Conclusion

By enforcing a strict global lock order across Account Database, Customer Info Database, and Transaction Logs, the bank server structurally removes the circular-wait condition, preventing this class of deadlock from recurring while keeping the system's concurrency largely intact.

**QUESTION 3***Mobile OS — App Switching Lag***10 MARKS****Original Question****Question 3 — Mobile OS App Switching Lag**

A mobile operating system is designed to support smooth app switching. But users report that:

- Opening WhatsApp delays the background music player.
- Switching between social media apps feels laggy.
- CPU shows high idle times despite multiple running apps.

The OS currently uses Round Robin (quantum = 20 ms) with 20 background apps active.

Tasks

1. Analyze why the CPU remains idle despite many active processes.
2. Determine whether the time quantum is helping or hurting performance.
3. Recommend a better scheduling method or quantum value, with justification.
4. Explain how the change will reduce switching lag and improve system throughput.

(10 Marks)

ANSWER**Aim**

To diagnose why a Round Robin mobile scheduler with a 20 ms quantum and 20 active apps produces both CPU idling and visible app-switching lag, and to recommend a fix.

Problem Understanding

Two symptoms that look contradictory actually share one root cause: most of the 20 “active” apps are I/O-bound / event-driven (waiting on network, sensors, or user input) rather than CPU-bound, so the CPU is frequently idle overall; but the plain Round Robin queue treats every app identically regardless of how interactive or time-sensitive it is, so a foreground-interactive app (WhatsApp notification, music buffer refill) can still be stuck waiting behind a full cycle of the ready queue.

Given Data

Parameter	Value
Scheduling Algorithm	Round Robin
Time Quantum	20 ms
Active Processes (queue length)	20
Worst-case cycle length (if all use full quantum)	$20 \times 20 \text{ ms} = 400 \text{ ms}$

Theory & OS Concept Used**Round Robin (Quantum = 20 ms)**

RR is a preemptive time-sharing algorithm; each process gets a fixed time slice before being placed at the back of the ready queue.

Working Principle: The ready queue is a circular FIFO; a process runs for at most one quantum, then is preempted if unfinished.

Advantages: Fair CPU share; Good response time; No starvation.

Disadvantages: Sensitive to quantum size; High context-switch overhead for small quanta.

Applications: Time-sharing OS, interactive terminals.

Detailed Explanation — Why the CPU is Idle Despite Many Apps

Most background apps spend the overwhelming majority of their time blocked in the Waiting state (waiting for network responses, push notifications, sensor events, or timers) rather than the Ready/Running state. The scheduler only measures whether processes exist, not whether they actually need the CPU right now — so total CPU-busy time can be low (high idle %) even while 20 process entries exist in the process table.

Scheduling / Quantum Analysis

The current $20 \text{ ms} \times 20$ -process configuration is hurting, not helping, performance. In the worst case where several apps use their full slice, a process at the back of the ready queue can wait up to $20 \times 20 \text{ ms} = 400 \text{ ms}$ before its next turn. For a foreground-interactive task like a WhatsApp UI update or a music player's audio buffer refill (which needs a fresh buffer every few tens of milliseconds to avoid glitching), a 400 ms worst-case wait is easily perceptible as lag or audio stutter.

⚠ Important: Round Robin with a large quantum and many processes degrades toward FCFS-like worst-case latency for any single process, which is exactly wrong for a UI-driven, interactive mobile workload.

Recommendation

Recommended fix: replace the flat Round Robin with a Multilevel Feedback Queue (MLFQ) that separates the visible foreground app from background apps, combined with a much smaller quantum for the interactive tier:

Queue Tier	Contains	Quantum	Priority
Tier 0 — Foreground/Interactive	Currently visible app (e.g., WhatsApp), active audio playback thread	2–5 ms	Highest
Tier 1 — Recently backgrounded	Apps switched away from in the last few seconds	10 ms	Medium
Tier 2 — Idle background	The remaining background apps awaiting events/notifications	20 ms (rarely used since mostly Waiting)	Lowest

Step-by-Step Solution

1. Classify each process by observed behaviour: an app that frequently blocks for I/O or user input is kept in / promoted to Tier 0; an app that consumes a full quantum repeatedly (CPU-bound behaviour) is demoted to a lower tier.
2. The scheduler always services Tier 0 first; Tiers 1 and 2 only run when Tier 0 has nothing ready, which is most of the time since Tier 0 apps are usually waiting on events rather than saturating the CPU.
3. Give the audio/music player thread a reserved minimum share (or place it permanently in Tier 0) so its periodic buffer-refill need is met regardless of what the visible foreground app is doing.

Performance Analysis

With Tier 0 quantum reduced to ~2–5 ms and only a small number of genuinely interactive tasks competing at that tier, worst-case foreground latency falls from ~400 ms to a few milliseconds — well within the ~16–20 ms budget needed for perceptibly smooth UI/audio. Overall CPU utilization is unaffected (still dictated by how much real work exists) but **useful**, user-visible

throughput and responsiveness improve sharply because CPU time is no longer wasted giving equal-length turns to apps that are mostly idle anyway.

Limitations

MLFQ is more complex to implement and tune (number of tiers, quantum per tier, promotion/demotion rules) than plain Round Robin, and an incorrectly tuned demotion rule could still let a misbehaving background app monopolize Tier 0 temporarily.

Real-world Applications

Android's process-priority/OOM-adjustment scheduling, iOS's App Nap / background execution limits, and general-purpose desktop OS schedulers (Linux CFS, Windows scheduler) all use similar foreground-priority, feedback-based ideas.

Result

The idle-CPU-with-lag symptom is explained by many I/O-bound apps combined with an over-long RR cycle; replacing flat RR with an MLFQ using a short (2–5 ms) foreground quantum resolves it.

Conclusion

Switching lag on this mobile OS is not a raw CPU-capacity problem (CPU is mostly idle) but a scheduling-fairness problem — a large flat Round Robin quantum with 20 competing apps creates unacceptable worst-case latency for interactive tasks; a feedback-based, foreground-prioritized scheduler resolves the lag while preserving background app responsiveness.



QUESTION 4

OS Update — Deadlock Risk

10 MARKS

Original Question

Question 4 — Deadlock Risk in an Operating System Update

During a Windows OS update, several modules run in parallel:

- M1: Copy update files
- M2: Install drivers
- M3: Update registry
- M4: Validate system integrity

Resources involved:

- R1 = Disk I/O
- R2 = Kernel lock
- R3 = Configuration lock

Recently logs show:

- M1 holds R1, waiting for R2
- M2 holds R2, waiting for R3
- M3 holds R3, waiting for R1

Tasks

1. Identify whether this sequence can lead to a deadlock.
2. If yes, explain the exact circular wait condition.
3. Propose Banker's algorithm style resource allocation to avoid risk.
4. Suggest how OS designers can prioritize modules to improve CPU utilization during update.

(10 Marks)

ANSWER

Aim

To determine whether the Windows update module dependencies (M1–M3 on R1–R3) can deadlock, and to design a Banker's-Algorithm-style avoidance scheme plus a module-priority strategy.

Problem Understanding

M1, M2, and M3 each hold one resource and request another in a rotating pattern, while M4 (validation) waits for all three to complete but does not yet hold or request any of R1–R3.

Given Data

Module	Holds	Requests
M1 — Copy update files	R1 (Disk I/O)	R2 (Kernel lock)
M2 — Install drivers	R2 (Kernel lock)	R3 (Configuration lock)
M3 — Update registry	R3 (Configuration lock)	R1 (Disk I/O)
M4 — Validate system integrity	— (not yet started)	Waits for M1–M3 to finish

Deadlock Analysis — Circular Wait

Process	Relationship	Resource	Edge Type
M1	holds →	R1	Assignment edge (Resource → Process)
M1	requests →	R2	Request edge (Process → Resource)
M2	holds →	R2	Assignment edge (Resource → Process)
M2	requests →	R3	Request edge (Process → Resource)
M3	holds →	R3	Assignment edge (Resource → Process)
M3	requests →	R1	Request edge (Process → Resource)

Yes, this sequence can lead to a deadlock. The exact circular wait: M1 is blocked waiting for R2, which is held by M2; M2 is blocked waiting for R3, which is held by M3; M3 is blocked waiting for R1, which is held by M1. This closes the cycle $M1 \rightarrow R2 \rightarrow M2 \rightarrow R3 \rightarrow M3 \rightarrow R1 \rightarrow M1$, and since each resource (Disk I/O, Kernel lock, Configuration lock) is a single-instance,

non-shareable resource during the update, mutual exclusion, hold-and-wait and no-preemption also hold — satisfying all four Coffman conditions.

Wait-for Graph

Waiting Process	Waits For (Holder of needed resource)
M1	M2
M2	M3
M3	M1

Recommended Technique — Banker's-Algorithm-Style Allocation

Banker's Algorithm (Deadlock Avoidance)

The Banker's Algorithm is a deadlock-avoidance technique that grants a resource request only if the resulting state is a safe state, i.e. a state from which every process can still finish given the resources available.

Working Principle: Before granting a request, the OS runs a Safety Algorithm: it checks whether there exists some ordering of processes such that each process's remaining Need can be satisfied by the currently Available resources plus the resources released by processes that finish before it.

Advantages: Avoids deadlock without restricting concurrency as much as strict prevention; Formally guaranteed safety if the algorithm always finds a safe sequence.

Disadvantages: Requires advance knowledge of each process's maximum resource need; Computationally more expensive than simple prevention rules; Not practical when the number of processes/resources is dynamic or very large.

Applications: Operating systems and databases with a bounded, known set of reusable resources.

To apply Banker's-Algorithm-style avoidance, the update installer declares each module's maximum resource need in advance and the OS grants a request only if the resulting state remains safe. Assume single instances of R1, R2, R3 (Available = 0,0,0 once fully allocated) plus M4, which needs all three only transiently to validate:

Module	Allocation (R1,R2,R3)	Max Need (R1,R2,R3)	Need (Max–Alloc)
M1	1,0,0	1,1,0	0,1,0
M2	0,1,0	0,1,1	0,0,1
M3	0,0,1	1,0,1	1,0,0
M4	0,0,0	1,1,1	1,1,1

Available = (0,0,0) since R1, R2 and R3 are all currently allocated. Running the Safety Algorithm: no module's Need can be fully satisfied by Available = (0,0,0), so this state is already unsafe — the Banker's Algorithm would never have granted the third of these three overlapping requests in the first place. Concretely, once M1 was granted R1 and M2 was granted R2, the OS should have DEFERRED granting R3 to M2 (or R1 to M3) because doing so leaves no safe completion order; the request would be queued until enough resources are free to guarantee a safe sequence.

Applying this avoidance rule retrospectively: a safe sequence exists only if requests are granted so that at least one module can always finish and release its resources next — e.g., grant M1 both R1 and R2 first (M1 finishes, releasing R1 and R2), then M2 can obtain R2 and R3 (M2 finishes, releasing them), then M3 obtains R3 and R1, and finally M4 validates using all free resources. Safe sequence: M1 → M2 → M3 → M4.

Step-by-Step Solution

1. Declare Max resource need for every module before the update begins (installer manifest).
2. On each resource request, run the Safety Algorithm against current Allocation and Available.
3. Grant the request only if a safe sequence still exists afterward; otherwise queue the requesting module.
4. Repeat until all modules (M1–M4) complete in a safe order, as shown above.

Module Prioritization for CPU Utilization

OS designers can prioritize modules to improve CPU utilization during the update as follows:

- Order modules by dependency, not arrival: schedule Disk I/O-heavy modules (M1) first since they are typically the long pole, and let CPU-bound modules (M3 — registry updates) run concurrently once their specific dependency is free rather than idling.
- Give M4 (validation) the lowest scheduling priority since it structurally cannot start until M1–M3 finish, avoiding wasted context switches to a module that will just re-block.

- Use non-blocking / asynchronous I/O for Disk I/O (R1) so the CPU can be handed to another ready module (e.g., M2 installing drivers) while M1 waits on the physical disk, instead of the CPU idling during I/O wait.

Performance Analysis

Under Banker's-style avoidance with the safe sequence $M1 \rightarrow M2 \rightarrow M3 \rightarrow M4$, all four modules complete without ever blocking indefinitely, versus the original ordering, which deadlocks and requires a full update rollback — a much larger real-world cost than the modest scheduling overhead of the safety check.

Limitations

Requires each module to accurately declare its maximum resource need in advance, which can be difficult for complex installers with conditional logic; the safety check itself adds a small but non-zero overhead on every resource request.

Real-world Applications

Operating system update/patch installers, package managers with dependency locking, and database migration tools that touch multiple shared resources.

Result

The $M1-M3$ pattern does lead to deadlock via the circular wait $M1 \rightarrow R2 \rightarrow M2 \rightarrow R3 \rightarrow M3 \rightarrow R1 \rightarrow M1$. Banker's-Algorithm-style avoidance, using the safe sequence $M1 \rightarrow M2 \rightarrow M3 \rightarrow M4$, prevents it.

Conclusion

A naive parallel-module update design creates a classic three-way circular wait; declaring resource needs in advance and granting requests only when a safe completion sequence exists (Banker's Algorithm) removes the deadlock risk while a dependency-aware module priority scheme keeps CPU utilization high during the update.



QUESTION 5

Cloud Server — CPU Utilization

10 MARKS

Original Question

Question 5 — Cloud Server CPU Utilization Analysis

A cloud hosting provider notices:

- 80% of the time, VM processes wait for network or disk
- Only 20% of the time is spent on CPU
- Each server runs 7 VMs

Using the formula: $U = 1 - p^n$ where p = I/O wait probability

Tasks

1. Calculate CPU utilization.
2. Interpret whether the server is underutilized or optimally loaded.
3. Suggest how increasing or decreasing number of VMs affects throughput.
4. Propose two OS-level strategies for improving CPU utilization in this cloud environment.

(10 Marks)

ANSWER

Aim

To calculate CPU utilization for a 7-VM cloud server using the probabilistic multiprogramming model and to recommend OS-level improvements.

Problem Understanding

Each VM independently spends 80% of its time waiting on I/O ($p = 0.8$) and only 20% actually using the CPU; with $n = 7$ VMs resident, the CPU is idle only when all 7 are simultaneously I/O-bound.

Given Data

Parameter	Value
I/O wait probability (p)	0.8 (80%)
CPU-active probability per VM	0.2 (20%)
Degree of multiprogramming (n)	7 VMs

Calculations — CPU Utilization

Formula: $U = 1 - p^n$

Substitution: $U = 1 - (0.8)^7$

$$(0.8)^7 = 0.8^4 \times 0.8^3 = 0.4096 \times 0.512 = 0.2097152$$

$$U = 1 - 0.2097152 = 0.7902848$$

CPU Utilization $\approx$ 79.03%

CPU Utilization Analysis / Interpretation

Degree of Multiprogramming (n)	CPU Utilization
4	59.04%
6	73.79%
7 (current)	79.03%

At $n = 7$, the server is well utilized — close to the point of diminishing returns — rather than underutilized. It is not yet saturated (100%), so there is still some headroom, but the marginal utilization gain per additional VM is shrinking (roughly +5 points going from 6→7 VMs versus +15 points going from 4→6), so the server is approaching optimal loading for this workload's I/O-wait profile rather than sitting idle.

Impact of Increasing/Decreasing VM Count on Throughput

Increasing VMs (e.g., $7 \rightarrow 10$): raises CPU utilization further (it becomes statistically less likely all VMs are I/O-blocked at once) and can raise throughput — but only up to the point where memory, network bandwidth, or disk queueing become the new bottleneck; beyond that, contention and increased context-switch overhead can make per-VM response time worse even as aggregate CPU utilization stays high (a form of thrashing).

Decreasing VMs (e.g., $7 \rightarrow 4$): lowers CPU utilization (idle CPU time rises to about 41%) and generally lowers aggregate throughput, but each individual VM gets more consistent,

lower-latency access to the CPU when it does need it — a useful trade-off for latency-sensitive workloads.

Recommended OS-Level Strategies

1. Increase the safe degree of multiprogramming using memory ballooning / overcommit and asynchronous, interrupt-driven or event-based I/O (instead of the CPU busy-waiting or blocking synchronously on network/disk calls), so that whenever one VM blocks for I/O the hypervisor's scheduler can immediately dispatch another ready VM, keeping the CPU busy.
2. Use a fair, credit-based VM/CPU scheduler with I/O-boosting (e.g., Xen's credit scheduler or similar) that temporarily raises the scheduling priority of a VM right after it returns from an I/O wait, so I/O-bound VMs are serviced promptly without starving CPU-bound ones — this both improves perceived responsiveness and keeps overall CPU utilization high.

Performance Analysis

Moving from 4 to 7 VMs raised utilization from 59.04% to 79.03%; adding OS-level asynchronous I/O and a credit-based scheduler on top of this can push realized utilization even closer to the theoretical maximum without a proportional loss in per-VM responsiveness.

★ **Exam Tip:** $U = 1 - p^n$ always increases with n because a higher-power term makes “all n VMs I/O-waiting simultaneously” less likely — remember this monotonic relationship for quick sanity-checks in the exam.

Limitations

Utilization gains from adding VMs diminish and eventually reverse once memory, network, or disk contention dominates; overcommitting VMs without matching I/O/memory capacity can cause thrashing and paradoxically reduce effective throughput despite a high CPU-utilization number.

Real-world Applications

Public cloud hypervisor scheduling (AWS/Azure/GCP), container orchestration CPU-share tuning (Kubernetes), and virtualization capacity-planning exercises.

Result

CPU Utilization at $n = 7$ VMs is approximately 79.03%, indicating a well- (not under-) utilized server.

Conclusion

The server is operating efficiently near the productive end of the multiprogramming curve; further VM increases should be paired with asynchronous I/O and I/O-aware scheduling to keep converting idle CPU capacity into useful throughput without tipping into resource contention.



QUESTION 6

Large-Scale CPU Scheduling Scenario

10 MARKS

Original Question

Question 6 – CPU Scheduling Scenario

Scenario:

A large-scale CPU scheduling environment in organization 1 experiences performance and reliability issues during peak usage. Multiple processes/resources interact simultaneously, causing delays, contention, and reduced throughput.

1. Analyze the root cause of the observed behavior.
2. Determine the most appropriate Operating System technique/algorithm to address the issue.
3. Justify your recommendation with respect to CPU utilization, throughput, response time, or fairness.
4. Design an improved system configuration or execution sequence for the given scenario.
5. Evaluate one limitation or trade-off of your proposed solution.

(10 Marks)

ANSWER

Aim

To diagnose the root cause of peak-time performance and reliability degradation in a large-scale, multi-process CPU scheduling environment and to design an improved scheduling configuration.

Problem Understanding

“Delays, contention, and reduced throughput” appearing specifically during peak usage — rather than being constant — points to a scheduling/resource-management issue that is fine at low load but breaks down once the number of concurrently ready processes exceeds what the current policy handles gracefully.

Detailed Explanation — Root Cause Analysis

Three compounding root causes are typical of this symptom pattern and should all be checked:

- Scheduling-policy mismatch: a simple algorithm such as FCFS or a single flat queue causes a convoy effect — long CPU-bound jobs block many short, latency-sensitive requests once the arrival rate spikes at peak.
- Resource contention: shared locks/critical sections (databases, shared data structures) become bottlenecks as more processes compete simultaneously, increasing effective waiting time even when raw CPU cycles are available.
- Excessive context-switching overhead: if too many processes are admitted without an appropriate quantum/priority structure, the fraction of CPU time spent on switching (rather than useful work) grows, silently eating into throughput exactly when it is most needed.

 **Key Concept:** Peak-only symptoms (fine at low load, broken at high load) almost always point to a scheduling or contention bottleneck rather than a hardware capacity limit — look at policy and locking behaviour before asking for more servers.

Recommended OS Technique

Multilevel Feedback Queue (MLFQ) Scheduling

MLFQ uses several ready queues ranked by priority; a process moves between queues based on its observed CPU-usage behaviour, allowing the scheduler to separate interactive (I/O-bound) processes from CPU-bound processes automatically.

Working Principle: New/interactive processes start in a high-priority queue with a short quantum; a process that uses its full quantum repeatedly is demoted to a lower-priority queue with a longer quantum, while a process that frequently blocks for I/O is kept at (or promoted back to) a high-priority queue.

Advantages: Automatically favours short/interactive jobs without a-priori knowledge of burst time; Balances responsiveness and throughput; Configurable to approximate almost any other scheduling policy.

Disadvantages: Complex to configure (number of queues, quantum per queue, promotion/demotion rules); Can still starve long CPU-bound jobs without aging.

Applications: General-purpose desktop and mobile operating systems; Multi-user time-sharing systems.

Recommendation: adopt a Multilevel Feedback Queue combined with load-based admission control (a bounded number of processes are allowed into the “actively scheduled” set at any

moment; excess arrivals are queued rather than all being made instantly Ready, preventing thrashing-style overload).

Justification

Metric	Effect of MLFQ + Admission Control
CPU Utilization	Improves — short/interactive jobs are serviced without long jobs blocking the CPU, and admission control avoids sched-overhead-dominated states
Throughput	Improves — fewer wasted context switches per unit of useful work completed
Response Time	Improves for short/interactive jobs, which are promoted to high-priority queues
Fairness	Maintained via aging/promotion rules that prevent long-term starvation of lower-tier jobs

Design — Improved System Configuration

Process arrives
↓
Admission control: is system below peak-load threshold?
↓
Yes → enter Ready queue at Tier 0 (short quantum)
↓
No → hold in Admission Queue until a slot frees
↓
Process exceeds quantum repeatedly? → demote to lower tier (longer quantum)
↓
Process blocks frequently for I/O? → keep/promote to higher tier
↓
Aging: raise priority of any process waiting too long in a lower tier

Performance Analysis

Because MLFQ automatically separates interactive from CPU-bound behaviour without needing advance knowledge of burst times, and admission control caps how many processes actively contend at once, both response time for short jobs and overall throughput improve during peak load, while aging preserves fairness for longer-running background work.

Limitations / Trade-off

The main trade-off is configuration complexity and tuning risk: choosing the number of tiers, per-tier quantum, promotion/demotion thresholds, and the admission-control threshold requires careful workload profiling; a poorly tuned admission-control threshold can itself introduce artificial queuing delay for legitimate requests during a genuine, sustained traffic increase (not just a scheduling artifact).

Real-world Applications

General-purpose OS kernels (Linux, Windows), application servers under load shedding/admission control, and large multi-tenant cloud platforms.

Result

Root cause: scheduling-policy mismatch and unmanaged contention/context-switch overhead under peak load. Recommended fix: Multilevel Feedback Queue with load-based admission control.

Conclusion

Peak-time degradation in a large-scale scheduling environment is best addressed not by adding raw capacity alone but by adopting an adaptive, feedback-based scheduling policy paired with admission control, which protects throughput and response time under load at the cost of additional tuning complexity.

**QUESTION 7***E-Commerce Platform — Process Management***10 MARKS****Original Question****Question 7:****Scenario:**

A leading international e-commerce company operates across 70 countries and processes nearly 10 million customer requests every day. The system consists of multiple activities such as user authentication, product search, shopping cart updates, payment processing, inventory management, and shipment tracking. During festive sales, the response time of the application increases significantly due to the large number of simultaneously executing tasks. The Chief Technology Officer (CTO) has formed a task force to investigate how the operating system manages these activities as processes and how process state transitions affect overall system performance. The team is also interested in understanding the role of context switching and Process Control Blocks (PCB) in maintaining efficient execution.

Questions:

1. Analyze the various processes involved in the given scenario.
2. Differentiate between a program and a process with suitable examples from the case study.
3. Explain how Process Control Blocks (PCB) are utilized by the operating system.
4. Illustrate the different process states that a payment transaction may undergo.
5. Evaluate the impact of context switching during peak sale periods.
6. Predict the consequences of excessive process creation in the system.
7. Design an efficient process hierarchy for the organization.
8. Justify the need for multiprogramming in this environment.
9. Recommend strategies to improve process management and scalability.
10. Develop a process lifecycle model suitable for the e-commerce platform.

(10 Marks)

ANSWER

Aim

To analyze how process management concepts — processes vs. programs, PCBs, process states, context switching, and process hierarchy — apply to a global e-commerce platform, and to recommend improvements for peak-load scalability.

Problem Understanding

A single logical operation (“checkout an order”) is really composed of several cooperating OS-level processes/threads (auth, search, cart, payment, inventory, shipping) that must each be scheduled, tracked, and context-switched efficiently across millions of daily requests, with load spiking sharply during festive sales.

OS Concept Used

Process concept, Process Control Block (PCB), process state model, context switching, multiprogramming, and process hierarchy/creation.

Detailed Explanation — Answers to Sub-Questions 1–10

1. Processes Involved

#	Process
1	User Authentication Process
2	Product Search Process
3	Shopping Cart Update Process
4	Payment Processing Process
5	Inventory Management Process
6	Shipment Tracking Process

Each customer request typically spawns or reuses a lightweight worker (process or thread) dedicated to one of these six activities, coordinated by a higher-level order-orchestration service.

2. Program vs. Process

Aspect	Program	Process
Definition	A passive, static set of instructions stored on disk (e.g., the PaymentService binary/code)	An active instance of that program currently executing in memory with its own state
Example from case study	The single “Authentication.exe” application code deployed once	Thousands of concurrently running authentication process instances, one per logging-in customer, each with its own PCB, memory, and current state

3. Role of the Process Control Block (PCB)

PROCESS CONTROL BLOCK (PCB)
Process ID (PID)
Process State (New/Ready/Running/Waiting/Terminated)
Program Counter
CPU Registers
CPU Scheduling Information (priority, queue pointers)
Memory Management Information (base/limit, page tables)
Accounting Information (CPU used, time limits)
I/O Status Information (open files, allocated devices)

The OS creates one PCB per active request-handling process. The scheduler and dispatcher use the PCB to save and restore a process's exact execution context on every context switch (e.g., pausing one customer's payment process to run another's), track its current state, and hold bookkeeping data such as which database connections or files it has open — without the PCB, the OS could not correctly suspend and later resume millions of concurrent customer operations.

4. Process States of a Payment Transaction

	NEW	
	↓ (admitted)	
	READY	
	↑ (dispatch / time-out)	
	RUNNING	
(I/O or event wait) ↙		↘ (exit)
WAITING		TERMINATED
(event completion → Ready)		
Stage	State	
Payment request submitted	New	
Waiting for a free CPU/worker	Ready	
Validating card details	Running	
Waiting for the external bank gateway's response	Waiting	
Confirming transaction and writing receipt	Running	
Transaction complete, resources released	Terminated	
 Remember: The bank-gateway call is an I/O wait, not CPU work — a payment process must move to Waiting (not stay Running) during it, freeing the CPU for other customers' requests.		

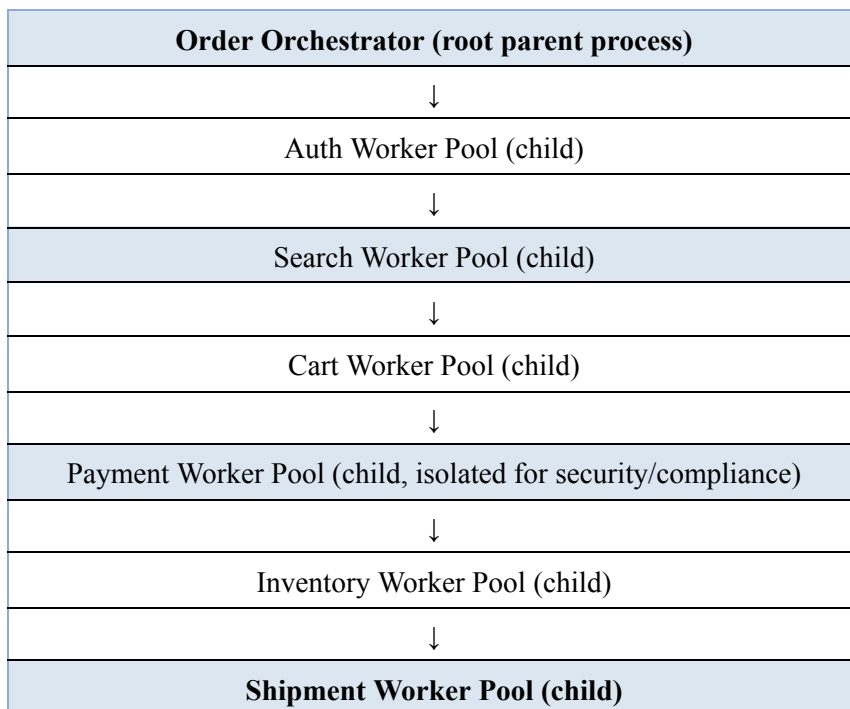
5. Impact of Context Switching During Peak Sale Periods

As concurrent request volume spikes during festive sales, the OS must context-switch between vastly more processes/threads in the same wall-clock time. Each switch has a fixed overhead (saving/restoring registers, program counter, memory-mapping/cache and TLB flush effects), so as switch frequency rises, a growing share of total CPU time is spent on overhead rather than useful transaction work — directly contributing to the increased response times the CTO's task force observed.

6. Consequences of Excessive Process Creation

- PCB/process-table exhaustion, risking new-request creation failures (effectively a self-inflicted denial of service).
- Rapidly increasing scheduler and context-switch overhead, degrading throughput even though “work” appears to be happening.
- Memory pressure and possible thrashing if per-process memory footprints (stacks, buffers) exceed available RAM.

7. Efficient Process Hierarchy Design



Rather than spawning a brand-new OS process per customer request (expensive to create/destroy), the design uses a small hierarchy of long-lived worker-thread pools per activity, supervised by an orchestrator parent — bounding total process count while still isolating each activity (particularly Payment, for security).

8. Justification for Multiprogramming

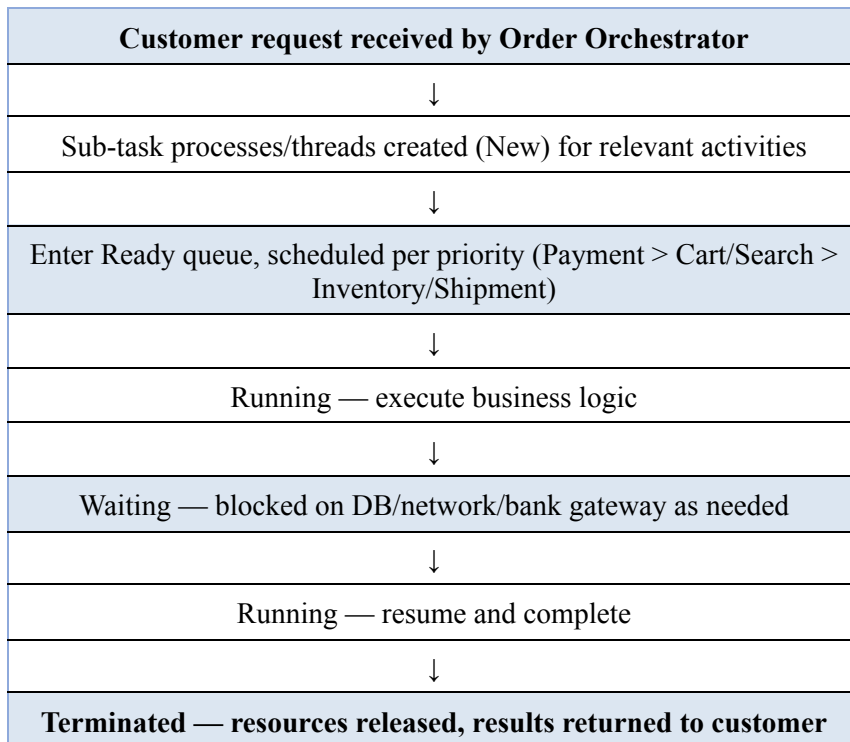
Multiprogramming keeps the CPU busy by switching to another ready process whenever the current one blocks — which is frequent here, since most of these activities (search, payment gateway calls, database writes) are I/O/network-bound. Without multiprogramming, the CPU would sit idle during every database or bank-gateway round trip; with it, the CPU serves other

customers' requests during those waits, which is essential to sustain 10 million requests/day across 70 countries.

9. Strategies to Improve Process Management and Scalability

1. Replace process-per-request with managed thread/worker pools to bound OS overhead.
2. Apply priority scheduling so payment-critical work is never starved by bulk/background tasks (e.g., analytics, recommendation refresh).
3. Use asynchronous, non-blocking I/O for external calls (bank gateways, warehouses) so worker threads are not held hostage waiting on network responses.
4. Horizontally auto-scale worker pools per activity based on real-time queue depth, especially ahead of known festive-sale peaks.

10. Process Lifecycle Model for the Platform



Performance Analysis

A hierarchy of bounded worker pools plus priority scheduling and asynchronous I/O keeps context-switch overhead proportional to genuine concurrency rather than raw request count, directly countering the peak-season response-time growth described in the scenario.

Limitations

Worker-pool sizing must be tuned per activity and per region (70 countries have different peak times), and incorrect pool sizing can either waste resources (oversized) or reintroduce queuing delay (undersized).

Real-world Applications

Large-scale e-commerce platforms (Amazon, Flipkart), payment gateways, and any high-concurrency web-service backend.

Result

A supervised worker-pool process hierarchy with priority scheduling, PCB-based context management, and asynchronous I/O directly addresses each of the ten investigation questions raised by the CTO's task force.

Conclusion

Understanding processes, PCBs, process states, and context switching allows the platform's process-management design to be re-engineered from an unbounded process-per-request model into a scalable, priority-aware worker-pool architecture that keeps festive-sale response times under control.



QUESTION 8

Smart Hospital — Process Scheduling

10 MARKS

Original Question

Question 8: Process Scheduling

Scenario:

A smart hospital has implemented an automated healthcare management system to handle patient registration, diagnostic services, medical record access, pharmacy management, and emergency response operations. During emergency situations, doctors have reported delays in retrieving patient records because CPU resources are being shared among multiple services. The hospital administration has requested the IT team to investigate different CPU scheduling policies and recommend a suitable scheduling strategy that prioritizes emergency services without starving routine tasks.

Questions:

1. Identify the scheduling challenges present in the hospital management system.
2. Compare FCFS, SJF, Priority Scheduling, and Round Robin for this application.
3. Analyze the impact of scheduling policies on emergency patient care.
4. Determine the most appropriate scheduling algorithm for the given scenario.
5. Evaluate the possibility of starvation and suggest remedies.
6. Recommend a suitable time quantum if Round Robin is selected.
7. Predict system performance during a mass casualty incident.
8. Justify your choice of scheduling algorithm using performance metrics.
9. Propose modifications to improve fairness and responsiveness.
10. Design a scheduling framework for the hospital's future expansion plans.

(10 Marks)

ANSWER

Aim

To evaluate CPU scheduling policy options for a hospital management system and recommend a strategy that prioritizes emergency record access without starving routine services.

Problem Understanding

Registration, diagnostics, medical-record access, pharmacy, and emergency-response all compete for shared CPU resources; the reported symptom (doctors delayed retrieving records during emergencies) suggests the current policy does not sufficiently differentiate urgent from routine requests.

Detailed Explanation — Answers to Sub-Questions 1–10

1. Scheduling Challenges Present

- No clear differentiation between emergency and routine service requests in the current design
- Risk of long routine batch tasks (e.g., bulk record indexing/reporting) blocking urgent record-access requests
- Need to bound worst-case response time for emergencies while still giving routine work a fair share

2. Comparison — FCFS vs SJF vs Priority vs Round Robin

Algorithm	Waiting Time	Emergency Responsiveness	Fairness	Starvation Risk	Suitability
FCFS	High (convoy effect possible)	Poor — no urgency awareness	High (order-based)	None	Not suitable alone
SJF	Low on average	Indirect only, if emergencies happen to be short	Low	Long jobs may starve	Not suitable alone (burst time $\neq$ urgency)
Priority Scheduling	Low for high priority	Excellent if emergency = highest priority	Low without aging	Routine tasks may starve	Best base algorithm

Algorithm	Waiting Time	Emergency Responsiveness	Fairness	Starvation Risk	Suitability
Round Robin	Moderate, evenly spread	Moderate — no urgency awareness	Highest	None	Good for the routine tier only

3. Impact of Scheduling Policies on Emergency Patient Care

Any policy that does not explicitly recognize urgency (FCFS, plain RR, plain SJF) risks life-threatening delay in retrieving a critical patient's record during an emergency, since such a request simply waits its turn alongside routine registration or pharmacy queries.

4. Most Appropriate Algorithm

A hybrid: Preemptive Priority Scheduling for triage-level urgency, implemented as a Multilevel Queue with Round Robin used only within the routine tier, and aging applied to the routine tier to guarantee it is never fully starved.

5. Starvation Evaluation and Remedies

⚠ Important: Under strict priority alone, routine tasks (registration, pharmacy) could be starved indefinitely if emergency requests arrive continuously. Remedy: apply aging — raise a routine task's effective priority the longer it waits — to guarantee an upper bound on its delay.

6. Recommended Time Quantum for the Routine Round Robin Tier

A moderate quantum (recommended: enough for roughly 80% of routine record/registration requests to complete within one or two slices, e.g., a quantum sized to the median routine request's processing time) balances responsiveness against context-switch overhead; too small a quantum wastes CPU time switching, too large approaches FCFS and reintroduces blocking delay for routine users.

7. Predicted Performance During a Mass Casualty Incident

If every incoming request is suddenly tagged “emergency,” the priority signal loses its discriminating power and routine tasks could be starved entirely, while emergencies themselves start queuing behind each other. Recommended mitigation: sub-tier emergency priority using a triage scale (e.g., Level 1 – immediately life-threatening, Level 2 – urgent, Level 3 – stable) so the scheduler can still differentiate within the emergency class, plus reserve a small guaranteed CPU-time share for routine operations even during a surge.

8. Justification Using Performance Metrics

Preemptive Priority (with triage sub-tiers) minimizes average and worst-case response time for the highest-urgency requests — the single most important metric for a hospital system — while aging bounds routine waiting time (fairness), and the RR sub-tier for routine tasks keeps their average waiting time low and predictable, together outperforming any single one of FCFS/SJF/RR/Priority used alone.

9. Modifications to Improve Fairness and Responsiveness

1. Introduce triage-based priority sub-tiers rather than a single “emergency” flag.
2. Apply aging to all non-top-tier queues.
3. Reserve a minimum guaranteed CPU-time percentage for routine services even under sustained emergency load.

10. Scheduling Framework for Future Expansion

Tier 0 — Life-critical (preemptive, triage sub-levels)
↓
Tier 1 — Urgent diagnostics/records (priority + short RR)
↓
Tier 2 — Routine registration/pharmacy (RR with aging)
↓
Central monitoring dashboard tracking response-time SLAs per tier
↓
Dynamic tier reassignment as request urgency is re-triaged

Performance Analysis

The multi-tier priority-with-aging design bounds worst-case response time for emergencies while guaranteeing routine services are never fully starved, which directly resolves the doctors' reported delay in retrieving patient records during emergencies.

Limitations

Requires an accurate, well-governed triage-priority assignment process — if urgency levels are mis-assigned (over- or under-triaged), the scheduler's guarantees no longer map to real clinical urgency.

Real-world Applications

Hospital information systems, emergency-dispatch computer-aided dispatch (CAD) systems, and any life-safety real-time computing environment.

Result

Preemptive, triage-tiered Priority Scheduling with aging for lower tiers is recommended for the hospital's CPU scheduling.

Conclusion

By explicitly modelling clinical urgency as scheduling priority (with triage sub-levels and aging), the hospital's system can guarantee fast emergency record access without indefinitely starving routine registration, diagnostics, or pharmacy operations.

**QUESTION 9***Smart City Traffic — IPC & Cooperating Processes***10 MARKS****Original Question****Question 9: Inter-Process Communication and Cooperating Processes****Scenario:**

A smart city traffic management system integrates traffic signals, CCTV cameras, weather stations, public transportation services, and emergency vehicle tracking systems. These independent components continuously exchange information to optimize traffic flow and reduce congestion. However, communication failures between processes have occasionally resulted in delayed traffic signal updates and emergency response times. The city administration seeks an improved communication model to ensure uninterrupted coordination among all subsystems.

Questions:

1. Identify the cooperating processes involved in the smart city ecosystem.
2. Analyze the communication requirements among these processes.
3. Compare shared memory and message passing techniques for this environment.
4. Recommend the most suitable IPC mechanism and justify your choice.
5. Evaluate the impact of communication failures on public safety.
6. Propose a fault-tolerant IPC architecture for the city.
7. Predict potential challenges as the city expands its infrastructure.
8. Suggest methods to improve synchronization among cooperating processes.
9. Design a secure communication framework for the system.
10. Develop a scalability plan for future smart city implementations.

*(10 Marks)***ANSWER****Aim**

To design an inter-process communication (IPC) model for a smart city traffic system that reliably coordinates traffic signals, CCTV, weather stations, public transport, and emergency-vehicle tracking.

Problem Understanding

These are physically distributed, independently developed subsystems that must exchange frequent, often time-critical, small messages; the reported problem (delayed signal updates and emergency response) is a direct symptom of an unreliable or poorly synchronized IPC design.

Detailed Explanation — Answers to Sub-Questions 1–10

1. Cooperating Processes

#	Process
1	Traffic Signal Controller Process
2	CCTV Analytics Process
3	Weather Station Data Process
4	Public Transportation Tracking Process
5	Emergency Vehicle Tracking Process

2. Communication Requirements

- Low latency for signal-control commands and emergency alerts
- High reliability / guaranteed delivery for safety-critical messages (emergency vehicle priority requests)
- Frequent, small periodic messages from sensors (CCTV counts, weather readings)

3. Shared Memory vs. Message Passing

Aspect	Shared Memory	Message Passing
Speed	Very fast (no kernel copy) for co-located processes	Slower — network/kernel overhead per message
Best fit here	Local edge device combining, e.g., CCTV analytics with the co-located signal controller	Cross-intersection, city-wide, physically distributed subsystems
Synchronization complexity	Higher — needs explicit mutual exclusion for shared state	Lower — messages are naturally serialized by the channel
Fault isolation	Weak — a crash can corrupt shared state	Strong — a crashed process cannot directly corrupt another's memory

4. Recommended IPC Mechanism

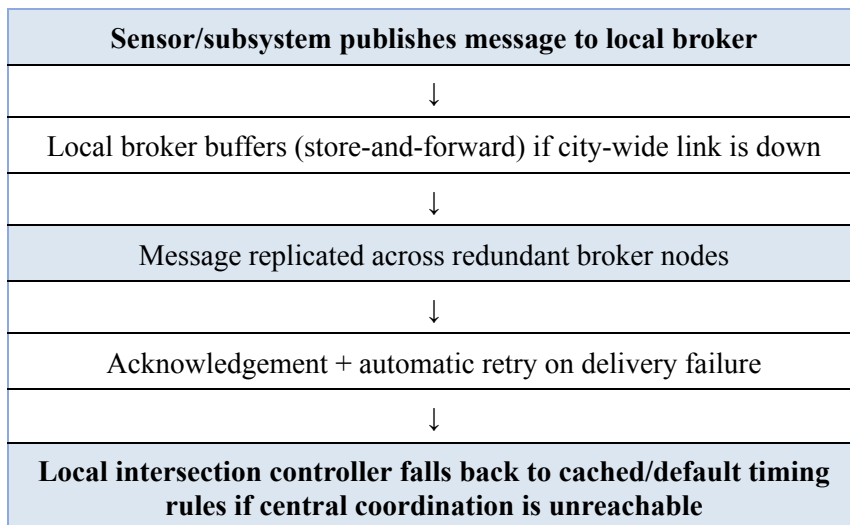
Message passing via a publish-subscribe message broker/queue is recommended as the primary city-wide mechanism, since subsystems are physically distributed across many intersections and

departments; shared memory is used only within tightly-coupled, co-located edge devices (e.g., a camera and its directly-attached local signal controller) where its speed advantage matters most.

5. Impact of Communication Failures on Public Safety

⚠ Important: A dropped or delayed emergency-vehicle-priority message can directly delay a signal change for an ambulance or fire truck, translating a software communication failure into a real-world public-safety and life-safety risk.

6. Fault-Tolerant IPC Architecture



7. Predicted Challenges as the City Expands

- Rapidly growing message volume as more sensors/cameras/intersections are added, straining broker capacity
- Increasing synchronization complexity across a larger number of cooperating processes
- Greater security/authentication overhead and a larger attack surface

8. Methods to Improve Synchronization

- Timestamp and sequence-number every message so consumers can detect and discard stale/out-of-order data
- Use semaphores/mutexes for any shared local state (e.g., a shared traffic-light timing table) and message queues for everything cross-process
- Adopt topic-based publish-subscribe routing so each subsystem only receives, and reacts to, messages relevant to it

9. Secure Communication Framework

Security Control	Purpose
TLS-encrypted channels	Protect message confidentiality/integrity in transit
Role-based access control per subsystem	Ensure, e.g., a CCTV process cannot issue signal-override commands
Digitally signed emergency-priority messages	Prevent spoofed emergency-override requests

10. Scalability Plan

Tier 1 — Local edge nodes: low-latency intersection-level IPC (shared memory / local bus)
↓
Tier 2 — District message brokers: aggregate several intersections
↓
Tier 3 — City-wide cloud message bus: city-scale optimization and analytics
↓
Horizontal scaling of broker clusters as sensor/intersection count grows

Performance Analysis

Separating fast local shared-memory IPC (within an edge node) from resilient, store-and-forward message-passing IPC (across the city) directly targets the reported failure mode — delayed signal/emergency updates — by ensuring a temporary network interruption degrades gracefully rather than silently dropping safety-critical messages.

Limitations

Message-passing IPC has inherently higher latency than shared memory, and a fully fault-tolerant, redundant broker architecture adds operational and infrastructure cost that must be justified against the public-safety benefit.

Real-world Applications

Smart-city traffic management platforms, SCADA/industrial control systems, and IoT sensor-network coordination architectures.

Result

A tiered, publish-subscribe message-passing architecture (with local shared memory only at the edge) with store-and-forward fault tolerance and signed emergency messages is recommended.

Conclusion

By matching the IPC mechanism to each level of physical/logical distribution — shared memory locally, resilient message passing city-wide — and adding redundancy plus authentication, the smart city system can maintain reliable, secure coordination even as failures occur or the infrastructure scales up.

**QUESTION 10***Live Streaming Platform — Threads & Multithreading***10 MARKS****Original Question****Question 10: Threads and Multithreading Models****Scenario:**

A global media company streams live sporting events to millions of viewers worldwide. To improve user experience, the application uses separate threads for video decoding, audio synchronization, subtitle generation, buffering, and advertisement insertion. During high-demand events such as the FIFA World Cup final, users occasionally experience video freezing and synchronization issues. The software architects have been asked to investigate whether the current multithreading model is appropriate for handling massive workloads.

Questions:

1. Identify the threads involved in the streaming application.
2. Compare user-level and kernel-level threads for this environment.
3. Analyze the suitability of One-to-One, Many-to-One, and Many-to-Many models.
4. Evaluate the consequences of thread blocking during live broadcasts.
5. Recommend an appropriate multithreading model with justification.
6. Predict system behavior under a tenfold increase in user traffic.
7. Suggest techniques to improve thread synchronization.
8. Design a thread management strategy for uninterrupted streaming.
9. Evaluate the role of multithreading in enhancing application performance.
10. Develop a monitoring framework for detecting thread-related issues.

*(10 Marks)***ANSWER****Aim**

To evaluate the multithreading model used by a live video-streaming client/server pipeline and recommend a design that eliminates the reported freezing and synchronization issues under massive concurrent load.

Problem Understanding

Video decoding, audio sync, subtitles, buffering, and ad-insertion run as separate threads; freezing/sync problems during peak events (FIFA World Cup final) suggest that at least one thread is blocking in a way that stalls others, or that the threading model does not provide enough true parallelism/isolation under load.

Detailed Explanation — Answers to Sub-Questions 1–10

1. Threads Involved

#	Thread
1	Video Decoding Thread
2	Audio Synchronization Thread
3	Subtitle Generation Thread
4	Buffering Thread
5	Advertisement Insertion Thread

2. User-Level vs. Kernel-Level Threads

Aspect	User-Level Threads	Kernel-Level Threads
Managed by	User-space thread library	Operating system kernel
Context-switch cost	Very low (no kernel trap)	Higher (kernel involvement)
Blocking behaviour	One thread blocking (e.g., on I/O) can block the whole process if the kernel is unaware of other threads	Kernel can schedule another thread of the same process while one blocks
True parallelism on multi-core	Not by itself	Yes
Best fit here	Not ideal alone — blocking risk unacceptable for live streaming	Needed, at least for I/O-bound threads (buffering/network)

3. Suitability of One-to-One / Many-to-One / Many-to-Many

Model	Suitability for This Streaming App
Many-to-One	Unsuitable — a single kernel thread means one blocking thread (e.g., Buffering waiting on network) stalls the entire process, directly explaining the freezing symptom
One-to-One	Best fit — each of the 5 functional threads maps to its own kernel thread, so one blocking (e.g., Buffering) does not stall Video Decoding or

Model	Suitability for This Streaming App
	Audio Sync, and true multi-core parallelism is possible
Many-to-Many	A viable middle ground (multiplexes many user threads onto fewer kernel threads) but adds scheduler complexity not clearly needed for only 5 well-defined functional threads per stream

4. Consequences of Thread Blocking During Live Broadcasts

If the client were using a Many-to-One model, the Buffering thread blocking on a slow network read would also freeze Video Decoding and Audio Sync (since all share one kernel thread), producing exactly the freezing/sync symptoms reported — this is very likely today's root cause.

5. Recommended Multithreading Model

 **Key Concept:** One-to-One threading (client-side) is recommended so each functional thread (decode, audio, subtitle, buffer, ads) has its own independent kernel thread and cannot be stalled by another blocking — directly resolving the freezing symptom.

6. Predicted Behaviour Under a 10× Traffic Increase

On the server/CDN side, naively creating one OS thread per viewer connection under One-to-One would exhaust kernel resources at 10× scale; the server side should instead use an asynchronous, event-driven architecture with bounded worker-thread pools (a Many-to-Many-style approach at scale), while the client device keeps its small, fixed One-to-One thread set per stream.

7. Techniques to Improve Thread Synchronization

- Lock-free ring buffers between the Decode and Audio-Sync threads to hand off frames without blocking either side
- Condition variables to wake the Buffering thread only when the buffer drops below a threshold, avoiding busy-waiting
- Monotonic shared timestamps for reliable audio/video sync instead of relying on wall-clock reads from multiple threads

8. Thread Management Strategy for Uninterrupted Streaming

Dedicated thread pool per function (Decode / Audio / Subtitle / Buffer / Ads)
↓

Priority: Video Decode & Audio Sync given highest scheduling priority
↓
Buffering thread uses asynchronous non-blocking network I/O
↓
Under CPU/resource pressure: gracefully pause/drop the Ad-Insertion thread first, never Decode/Audio
↓
QoS monitor adjusts thread priorities dynamically based on buffer health

9. Role of Multithreading in Performance

Multithreading allows the decode → audio-sync → render pipeline stages to overlap in time (pipelining) rather than execute strictly sequentially, and lets independent stages (e.g., subtitle generation) run in parallel on separate CPU cores — reducing end-to-end latency and enabling smoother playback on modern multi-core devices.

10. Monitoring Framework for Thread-Related Issues

Metric Monitored	Purpose
Decode thread FPS / frame-drop rate	Detect decode-side stalls in real time
Audio buffer underrun count	Detect audio/video desync events
Buffering-thread network wait time	Detect network-induced blocking risk
Per-thread CPU time and scheduling delay	Detect starvation or priority-inversion issues

Performance Analysis

Moving from a Many-to-One-style vulnerability (one blocking thread stalling all others) to a One-to-One client model with asynchronous I/O directly removes the coupling that causes freezing, while server-side async worker pools let the platform absorb a 10× traffic surge without exhausting kernel thread resources.

Limitations

One-to-One threading has a per-thread kernel resource cost that does not scale to “one thread per viewer connection” on the server; that side must instead rely on event-driven/async architectures, which are more complex to implement correctly than naive per-connection threads.

Real-world Applications

Live sports/video streaming platforms (e.g., major OTT broadcasters), video-conferencing clients, and any real-time multimedia pipeline.

Result

A client-side One-to-One threading model (isolating Decode, Audio, Subtitle, Buffer, and Ad-Insertion) combined with asynchronous non-blocking I/O and a server-side async worker-pool architecture is recommended.

Conclusion

The observed freezing and sync issues during the FIFA World Cup final are best explained by blocking coupling between functional threads; adopting One-to-One threading on the client with non-blocking I/O, alongside a scalable async architecture on the server, resolves the freezing while preserving smooth audio/video synchronization under massive peak load.